

DESAFIO TRAINEES — CURSOR + ENGENHARIA DE CONTEXTO

Mais do que gerar código, o objetivo deste desafio é aprender a estruturar sistemas, organizar contexto e utilizar IA de forma profissional.

A IA não será avaliada pelo quanto ela escreveu código.

Ela será avaliada pelo quanto vocês conseguiram organizar o trabalho dela.

O Desafio

Vocês deverão desenvolver uma aplicação completa (frontend + backend) utilizando Cursor e IA como ferramenta principal de desenvolvimento.

O foco não é apenas “fazer funcionar”. O foco principal será:

- Arquitetura do sistema
- Organização de contexto
- Uso correto de documentação
- Uso correto de rules
- Separação de responsabilidades
- Escalabilidade da aplicação
- Clareza técnica
- Experiência do usuário
- Capacidade de evolução do projeto

Tema da Aplicação

A aplicação será um mini RPG corporativo chamado:

Corporate Survivor

O jogador começa como trainee e precisa sobreviver à primeira semana em uma empresa.

Cada decisão altera atributos do personagem:

- Energia
- Reputação
- Networking
- Ansiedade
- Produtividade
- Aprendizado

Fluxo obrigatório do jogo

- O jogador deve cadastrar seu nome ao iniciar
- O jogo deve possuir tela inicial
- O jogador inicia como trainee em seu primeiro dia
- O jogo acontece durante 5 dias
- Cada dia deve possuir exatamente 3 eventos principais
- O jogo deve possuir no mínimo 15 eventos
- Eventos secretos e desbloqueáveis são opcionais
- Ao final do jogo, o sistema deve mostrar o final atingido
- O sistema deve mostrar score final e ranking global

Como o jogo funciona

- Cada evento apresenta um cenário corporativo
- O jogador deve escolher uma entre várias opções
- Cada escolha altera os atributos do personagem
- Algumas escolhas desbloqueiam ou bloqueiam eventos futuros

- O jogo precisa reagir ao estado atual do jogador
- O sistema deve permitir múltiplos finais

Exemplo de evento

Seu gestor manda mensagem às 18h:

“Rapidinho, consegue subir isso hoje?”

- A) “Claro, pode deixar comigo.” → produtividade +2 / energia -2 / ansiedade +1
- B) “Consigo amanhã cedo com mais segurança.” → reputação +1 / produtividade -1 / ansiedade -1
- C) “Não entendi completamente o escopo.” → aprendizado +2 / reputação -1
- D) Ignorar a mensagem. → energia +1 / reputação -3

Engine do jogo

- Os eventos devem ser carregados dinamicamente via JSON/configuração
- Os eventos NÃO podem ser hardcoded diretamente na interface
- Cada evento deve possuir id, descrição, opções e consequências
- Os eventos devem possuir condições de desbloqueio
- As consequências devem alterar atributos do jogador
- O sistema deve permitir adicionar novos eventos sem alterar a lógica principal
- A lógica do jogo deve ser desacoplada da interface

Sistema de pontuação

O jogo deve possuir score final baseado nos atributos do jogador.

- Os atributos devem impactar a pontuação final
- O sistema deve possuir ranking global

- Os jogadores devem ser salvos no banco de dados
- O ranking deve exibir nome, score e final atingido

Regras obrigatórias do jogo

- O jogador deve possuir atributos persistentes
- As escolhas devem impactar o jogo de forma real
- Eventos futuros devem depender do estado do jogador
- O sistema precisa possuir save automático
- O usuário deve conseguir continuar uma partida salva
- O usuário deve conseguir reiniciar a jornada
- O jogo precisa possuir ao menos 3 finais diferentes

Condições de fim de jogo

- A semana termina (sexta-feira)
- Algum atributo chega a zero
- O personagem é demitido
- O personagem entra em burnout
- O personagem se torna destaque da equipe
- O personagem sobrevive normalmente ao onboarding

Exemplos de finais:

- Trainee Lenda
- Promessa Corporativa
- Funcionário Invisível
- Sobrevivente do Onboarding
- Risco Operacional

- Burnout em Tempo Recorde
- Demitido no Período de Experiência

Requisitos Técnicos Obrigatórios

- Frontend + Backend obrigatórios
- Banco de dados obrigatório: SQLite
- Persistência de estado obrigatória
- Salvar progresso, atributos e decisões do jogador
- Arquitetura desacoplada
- Código organizado e componentizado
- Projeto responsivo
- README completo
- Documentação obrigatória
- Uso obrigatório de rules no Cursor

Estrutura Esperada do Projeto

/frontend

/backend

/docs

architecture.md

game-rules.md

api.md

decisions.md

.cursor/rules

frontend.mdc

backend.mdc

game-engine.mdc

Importante

- Adicionar novos eventos NÃO deve exigir alterar múltiplos arquivos
- A lógica do jogo deve ser extensível
- O projeto deve permitir evolução futura
- A documentação deve permitir que outra pessoa entenda rapidamente o sistema
- As rules devem refletir decisões reais de arquitetura e padronização
- O sistema deve ser facilmente expansível para novos modos ou histórias

Objetivo Real do Desafio

Este desafio não é sobre criar apenas uma aplicação divertida.

O objetivo é aprender a trabalhar com IA da forma como empresas modernas estão começando a trabalhar:

- Estruturando contexto
- Criando sistemas sustentáveis
- Tomando decisões de arquitetura
- Definindo regras claras
- Documentando conhecimento
- Organizando colaboração entre humanos e IA

Boa sorte.

E lembrem-se:

Quanto maior o projeto, menos importante é “pedir código”.

E mais importante passa a ser organizar inteligência.